Advanced Time Series Forecasting: SARIMA, GARCH, Machine Learning

CEFET/RJ

Data Analytics Lab

dal.global@grupo.cefet-rj.br | https://eic.cefet-rj.br/dal

Regime Change as a Bridge to Event Detection

What are regime changes?

Many time series exhibit **structural breaks** — moments when the dynamics change abruptly:

Regime change

The process jumps between states with distinct parameters (μ_k, ϕ_k).

Change points

The structure of the series changes at a specific instant and remains different thereafter.

Drift

Gradual drift in parameters over time.

The limitation of classical models

> ARIMA, SARIMA, and GARCH assume **fixed parameters over time** — they are unable to represent dynamics that change regime.

When the series exhibits structural breaks, these models become misspecified: the residuals cease to be white noise and predictive power drops.

The detection of **change points, structural breaks, and drift** will be covered in depth in the dedicated block on **Event Detection**.

Limitations of Linear Models

ARIMA models describe linear temporal dependence and assume stationarity after differencing. They assume constant variance over time, but do not capture complex temporal structures.

Real-world series exhibit seasonality, nonlinearity, and varying variance. Linear models are a powerful but limited starting point. When a series presents richer patterns — such as cycles, variance shifts, or nonlinear relationships — we need to expand the model.

SARIMA: Seasonal Extension of ARIMA

ARIMA captures short-term dependency. SARIMA goes further by incorporating patterns that repeat every seasonal period — such as months or quarters. Both mechanisms operate in parallel within the same model.

> **SARIMA**$(p, d, q)(P, D, Q)(s)$

Regular scale

Captures dependency between nearby observations

Seasonal scale

Captures patterns that repeat every s periods

The result is a model that simultaneously describes the local dynamics and the periodic rhythm of the series.

Identifying and Fitting SARIMA Models

The SARIMA modeling process combines two interdependent steps: identifying the appropriate structure and fitting the model to the data.

Identification

Stationarize the series by applying regular differences (order d) and seasonal differences (order D):

$$Y_t = \Delta^d \Delta_s^D X_t$$

With Y_t stationary, analyze the ACF and PACF:

- **Regular lags** $(1, 2, \ldots)$ → orders p and q
- **Seasonal lags** $(s, 2s, \ldots)$ → orders P and Q
- **Information criteria** AIC and BIC to compare candidate models

Fitting and Diagnostics

With the orders defined, estimate the parameters and validate the model:

- **Estimation** by maximum likelihood
- **Ljung-Box test** to verify absence of autocorrelation in the residuals
- **Residual normality** (QQ-plot, Shapiro-Wilk test)
- **Final selection** of the model with the best balance between fit and parsimony

> The logic is the same as ARIMA — but applied at two temporal scales simultaneously.

Regression with Temporal Errors

When the series depends on external variables Z_t, the model separates two distinct mechanisms:

$$X_t = \underbrace{\beta_0 + \beta \cdot Z_t}_{\text{external effect}} + \underbrace{u_t}_{\text{temporal dependence}}$$

External Effect (Regression)

The term $\beta_0 + \beta \cdot Z_t$ captures the influence of exogenous variables on X_t. It is the part explained by factors external to the series.

Temporal Dependence (Error)

The term u_t **is not white noise** — it follows an ARMA or ARIMA process, capturing all remaining temporal structure after controlling for Z_t.

⚠ **Why is the separation fundamental?** Ignoring the temporal structure of u_t invalidates inference: OLS standard errors become biased and significance tests turn spurious. OLS is inconsistent in this setting — it is necessary to use **GLS** or **joint MLE** to correctly estimate the parameters.

Dynamic Models

An external variable can affect the series immediately or with a delay. The distributed lag model captures this effect over time:

$$X_t = \alpha + \beta_0 Z_t + \beta_1 Z_{t-1} + \ldots + \beta_k Z_{t-k} + u_t$$

Immediate Effect

Impact of $\beta_0 Z_t$ at the current time

Lagged Effects

Impacts of $\beta_1 Z_{t-1}$, $\beta_2 Z_{t-2}$, etc.

Temporal Propagation

Cumulative effects over time

SARIMAX: SARIMA with Regression

SARIMAX extends SARIMA by incorporating exogenous variables Z_t, separating two distinct effects into a single model:

$$X_t = \beta \cdot Z_t + u_t, \quad u_t \sim \text{SARIMA}(p,d,q)(P,D,Q)_s$$

External Effect

The term $\beta \cdot Z_t$ captures the direct impact of the exogenous regressor variables on the series

Serial Dependence

The error u_t follows a SARIMA structure, capturing the remaining internal temporal dynamics

Fundamental Separation

The regression explains external effects; the SARIMA explains the temporal structure of the residuals

Temporal Heteroscedasticity

In many series, the mean remains stable, but the variance fluctuates over time — forming alternating periods of low and high volatility. The conditional variance depends on past information:

$$\mathrm{Var}(X_t \mid \mathcal{F}_{t-1}) = \sigma_t^2$$

where $\mathcal{F}_{t-1}$ represents all information available up to time $t-1$. When σ_t^2 is not constant, the process is heteroscedastic.

Financial markets

Volatility concentrates during periods of crisis or uncertainty

Industrial sensors

Variance increases under extreme operating conditions

Demand and consumption

Greater dispersion on seasonal dates or atypical events

ARCH and GARCH models were developed to capture and forecast this variable variance structure.

ARCH/GARCH Estimation

Core Idea

The conditional variance is modeled as a function of past shocks and variances. GARCH generalizes ARCH by including memory of the variance itself.

ARCH:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i \varepsilon_{t-i}^2$$

GARCH(1,1):

$$\sigma_t^2 = \alpha_0 + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

For GARCH(1,1), the condition $\alpha_1 + \beta_1 < 1$ ensures variance stationarity.

```
# ARCH/GARCH Estimation
INPUT: ε_t (residuals from the mean model), orders (p, q)
PRE-CONDITION: significant ARCH test on ε_t

STEP 1 — Initialization
  σ²_1 ← sample variance of ε

STEP 2 — Conditional variance modeling
  FOR each t from 2 to T:
    σ²_t ← α_0 + Σ α_i · ε²_{t-i} + Σ β_j · σ²_{t-j}

STEP 3 — Parameter estimation
  Maximize conditional log-likelihood L(α, β | ε)

STEP 4 — Diagnostics
  z_t ← ε_t / σ_t  (standardized residuals)
  Check: z_t has no autocorrelation and no residual ARCH effect

STEP 5 — Forecasting
  [technical note] σ²_{T+h} converges to the unconditional variance
  at rate (α_1 + β_1)^h as h grows

OUTPUT: estimated sequence σ²_t for all t
POST-CONDITION: z_t approximately white noise
```

Volatility Persistence in GARCH

GARCH models the **conditional variance** — not the mean — capturing how uncertainty evolves over time. A shock in ε_t raises σ_t^2, and this effect propagates over several periods.

The degree of persistence is measured by $\alpha_1 + \beta_1$. The closer to 1, the more slowly volatility returns to its unconditional level:

$$\sigma^2_{T+h} = \bar{\sigma}^2 + (\alpha_1 + \beta_1)^h \left(\sigma^2_T - \bar{\sigma}^2\right)$$

where $\bar{\sigma}^2 = \alpha_0/(1 - \alpha_1 - \beta_1)$ is the long-run unconditional variance.

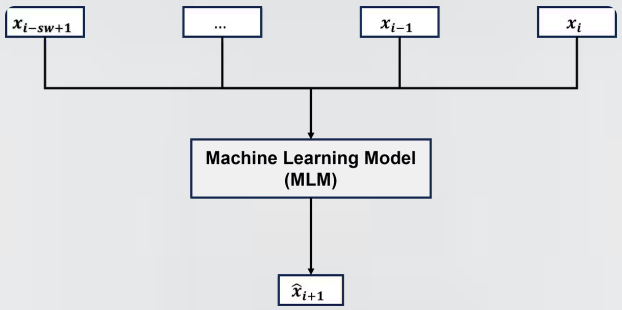
α + β small

Shock dissipates quickly. Confidence intervals return to the baseline level within a few periods.

α + β close to 1

High persistence. Uncertainty remains elevated — typical of crises. Confidence intervals widen for weeks.

This positions GARCH as a **risk forecasting** tool: it does not forecast the level of the series, but the width of future confidence intervals.



Machine Learning for Time Series

ML models are **universal approximators** — they can approximate any continuous function with arbitrary precision. Unlike statistical models, they do not require explicit specification of temporal structure: this structure is learned from the data through a sliding window of lagged observations.

The general input-output scheme is:

Input

Sliding window of p lagged values:

$$\langle x_{t-p+1}, \ldots, x_{t-1}, x_t \rangle$$

Output

Next predicted value:

$$\hat{x}_{t+1}$$

Hyperparameter

The number of lags p (window size) is defined as a hyperparameter and tuned via temporal cross-validation.

Neural Networks (MLP / ELM)

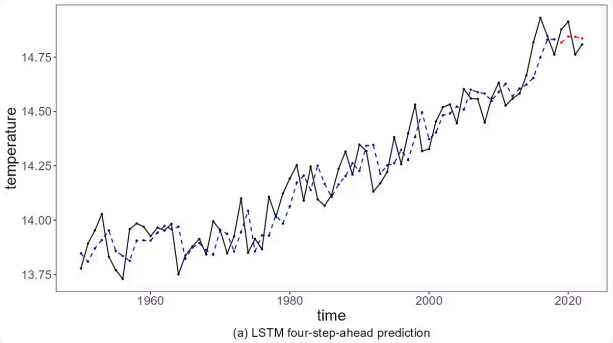
- MLP is the most common architecture for time series
- ELM replaces iterative gradient descent with analytical determination of weights
- ELM is significantly faster to train

SVM and Random Forest

- SVM maps inputs to a feature space induced by a kernel, enabling nonlinear regression
- Random Forest combines multiple decision trees
- Generalization error converges with sufficient trees, reducing overfitting

Deep Learning for Time Series

Deep learning extends classical neural networks by learning hierarchical representations of temporal data. Two architectures stand out for time series: Conv1D and LSTM.



(a) LSTM four-step-ahead prediction

Conv1D (1D Convolutional Networks)

- Composed of an input layer, convolutional layer, and pooling layer
- Convolution filters act as implicit feature extractors of the temporal signal
- Efficiently captures local temporal patterns

$$(g * h)[n] = \sum_{t=0}^{k} g[k-t] \cdot h[t]$$

$$a_n^{(l)} = \mathrm{ReLU}(z_n^{(l)}) = \max(0,\ z_n^{(l)})$$

$$p_j = \max_{n \in \mathcal{R}_j} a_n^{(l)}$$

where g is the input, h is one of the k filters optimized during training, $a_n^{(l)}$ is the activation at layer l, and p_j is the pooled value over window $\mathcal{R}_j$.

LSTM (Long Short-Term Memory)

The gates and cell state are governed by the equations:

$$\begin{pmatrix} f_t \\ i_t \\ o_t \end{pmatrix} = \sigma\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b \right)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C[h_{t-1}, x_t] + b_C)$$

$$h_t = o_t \odot \tanh(C_t)$$

where x_t is the input at time t, h_{t-1} is the previous hidden state, C_{t-1} is the previous cell state, σ is the sigmoid function, and $\odot$ is the element-wise product.

ML models — including deep learning — should be compared against statistical baselines (ARIMA, SARIMA) using temporal cross-validation. The added complexity is only justified when the gain in out-of-sample performance is consistent.

LLMs and Graph-Based Models

The success of Large Language Models (LLMs) in NLP inspired adaptations for time series. Both language models and time series models predict future patterns from sequential structure. In LLMs, the next token; in time series, the next value.

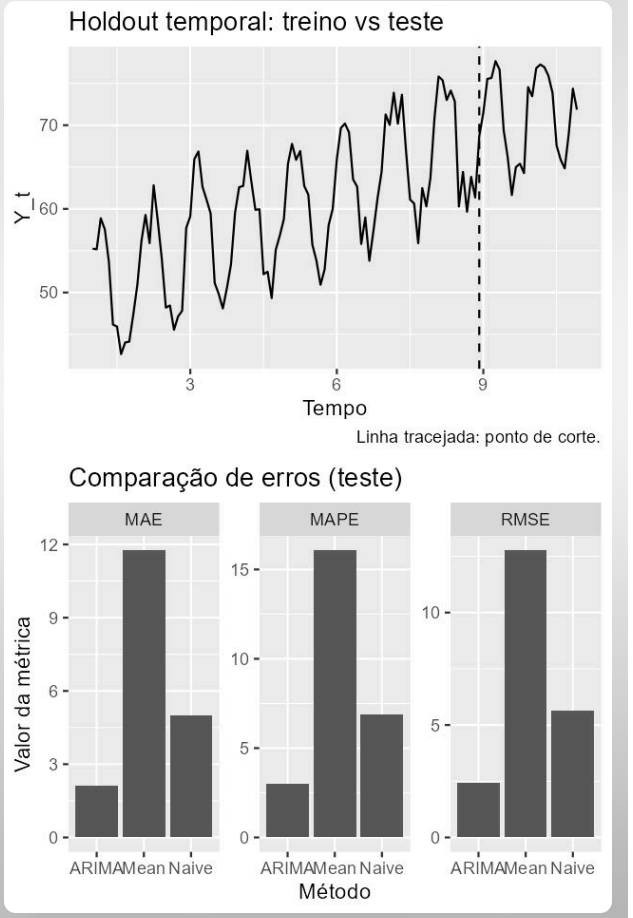
LLMs for Time Series:

- Time series values are mapped to tokens via scaling and quantization (discretization into bins)
- Pre-trained transformer models receive the tokenized series and predict the next token
- Output: probabilistic distribution over tokens → sampled → dequantized → predicted values
- Key advantage: zero-shot learning — LLMs pre-trained on diverse series can forecast new series without specific retraining
- Limitation: requires quantization design; may lose fine-grained numerical precision

Graph-Based Models (STGNNs):

- Represent dependencies among multiple time series as a graph: nodes = individual series, edges = pairwise relationships
- Spatiotemporal Graph Neural Networks (STGNNs) use message-passing to propagate temporal and spatial information
- Architectures: time-then-space (TTS), space-then-time (STT), time-and-space (T&S)
- Global models share parameters across all series (scalability); local models add node-specific embeddings (specificity)
- Applications: traffic forecasting, energy analysis, air quality monitoring

These are frontier extensions — they go beyond univariate forecasting and explore large-scale pre-training or relational structure among series.



Forecast Error Metrics

Error metrics allow us to compare models and evaluate predictive quality. Each metric emphasizes a different type of error, such as average magnitude, variability, or sensitivity to extreme values.

MSE - Mean Squared Error

$$\mathrm{MSE} = \frac{1}{n}\sum_{t=1}^{n} e_t^2$$

Strongly penalizes large errors

MAE - Mean Absolute Error

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n} |e_t|$$

More robust to outliers, treats all errors linearly

MAPE - Percentage Error

$$\mathrm{MAPE} = \frac{100}{n} \cdot \sum_{t=1}^{n} \left| \frac{e_t}{y_t} \right|$$

Expresses the error as a percentage of the observed value

Note: MAPE is unstable when y_t is close to zero.

Choosing the Loss Function

Choosing an error metric implicitly means choosing a loss function. If we use MSE, we seek the conditional mean; if we use MAE, we seek the conditional median. There is no single "best" forecast in all contexts.

Quadratic Loss

$$L(e) = e^2$$

Leads to the mean as the optimal forecast

Absolute Loss

$$L(e) = |e|$$

Leads to the median as the optimal forecast

Asymmetric Loss

Penalizes positive and negative errors differently

Temporal Validation

In time series, time is a constraint: the model can only use the past to predict the future. Validation must respect this order.

Rolling Window

Fixed training size; the window advances in time at each iteration.

Expanding Window

Training grows by accumulating all available history up to the current point.

```
INPUT: series x[1..T], window W, horizon h
PRE-CONDITION: T > W + h

STEP 1 — For each evaluation point t = W+1, ..., T−h:

  // Rolling window:
  train ← x[t−W .. t−1]

  // Expanding window (alternative):
  train ← x[1 .. t−1]

STEP 2 — Fit and forecast
  model ← fit(train)
  x̂[t+h] ← predict(model, h)

STEP 3 — Record error
  e[t] ← x[t+h] − x̂[t+h]

OUTPUT: errors e[W+1 .. T−h]
POST-CONDITION: no future data used in training
```

Statistical Model Comparison

Comparing models requires care. In time series, the order of data matters — and evaluation must respect that structure.

Same Test Set

Different models can only be compared if evaluated over the same period and with the same error metric.

A model that appears better on training data may perform worse on the test set. Valid comparison is always made outside the fitting period.

Overfitting in Time Series

Overfitting occurs when the model fits noise instead of true structure. It is a common problem in complex models and manifests when training performance is much better than test performance.

Main Symptom

$$\text{Erro}_{\text{treino}} \ll \text{Erro}_{\text{teste}}$$

Cause

Model captures random fluctuations from the past that do not repeat

Solution

Rigorous temporal validation and appropriate regularization

The Limits of Forecasting

There is always an unpredictable component in any time series. Even the best possible model has a residual error that cannot be eliminated. Part of the variability is inherent to the process and defines a theoretical limit for predictability.

Predictable Component

Systematic structure that can be modeled

Irreducible Noise

Random variability inherent to the process

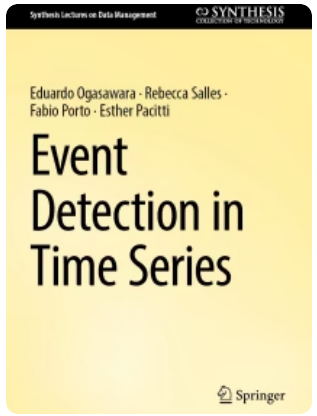
Time Horizon

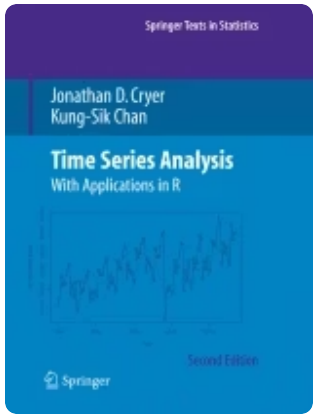
Predictability decreases over time

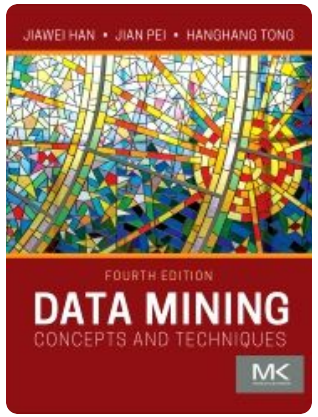
Decomposition

$$X_t = \hat{X}_{t|t-1} + \varepsilon_t$$

Bibliographic References







Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.; Pacitti, E. Springer Nature Switzerland, 2025.

Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan Kaufmann, 4th ed., 2022.